

Prof. Stefan Göller
Anna Maiworm

Einführung in die Informatik

WiSe 2025/2026

Hausaufgabenblatt 09, Abgabe am 19.01.26 um 12:00 Uhr

Allgemeine Hinweise

- Wenn die Hausaufgabe exakte Benennungen von Dateien, Funktionen, Variablen, etc. vorgibt, dann müssen Sie sich an diese Vorgabe halten. Falls nicht wird Ihre Abgabe nicht oder nur mit Punktabzügen korrigiert.
- Gleiches gilt, wenn die Aufgabe konkrete Anweisungen für `input()` oder `print()` vorgibt.
- Ihr Programm darf keine zusätzlichen `print()` Befehle enthalten, welche in der Aufgabe nicht gefordert sind. Ist eine Ausgabe mit `print()` gefordert, so geben Sie, sofern nicht anders vorgegeben, nur das geforderte Ergebnis und **keinen** zusätzlichen Text aus.
- Benutzen Sie **keine Python-Imports**, außer dies ist in der Aufgabenstellung explizit erlaubt.
- Beachten Sie die Anweisungen, die ggf. zusätzlich in den jeweiligen Aufgaben gegeben sind.
- Geben Sie für jede Aufgabe eine separate Datei ab. Benennen Sie diese Datei als `bXXaY.py`, wobei XX die Blattnummer und Y die Aufgabennummer ist.
- Schreiben Sie die Namen aller Gruppenmitglieder als Kommentar in die erste Zeile jeder Datei.
- Programmcode muss möglichst einfach und gut lesbar sein, um die Korrektur zu erleichtern. Code, der nicht oder nur schwer lesbar ist, kann zu Punktabzug führen.

Bitte beachten Sie, dass Hausaufgabe 3 ausnahmsweise im .pdf-Format und Aufgabe 4 teils im .pdf, teils im .py Format abzugeben ist.

Hausaufgabe 1 (2 + 2 = 4 Punkte):

- (a) Schreiben Sie eine Funktion `string_funk`, welche als Eingabe zwei Strings `s1` und `s2` erwartet, sodass `s1` mindestens die Länge 3 hat, `s1` doppelt so lang wie `s2` ist und `s1` und `s2` in den ersten 3 Positionen übereinstimmen. Zurückgegeben werden soll die Konkatination von `s1` und `s2`.

Verwenden Sie `assert`, um in der Funktion sicherzustellen, dass die Eingaben wie beschrieben sind.

- (b) Schreiben Sie eine Funktion `nutzer_aufruf`, welche keine Eingabe erwartet. Die Funktion fragt vom Nutzer zwei Strings an, ruft `string_funk` auf diesen auf und gibt, wenn möglich, das Ergebnis auf der Konsole aus. Mögliche auftretende Assertion-Errors sollen von der Funktion gefangen und die Fehlermeldung "Eingaben nicht zulässig." auf der Konsole ausgegeben werden.

Hausaufgabe 2 (6 Punkte):

Folgende Funktion soll überprüfen, ob eine gegebene Datei in der ersten Zeile ein valides Datum (TT.MM.YYYY) enthält.

```
def check_file(file: str) -> int:
    file = open(file, 'r')
    datetime.strptime(next(file), "%d.%m.%Y")
    return 0
```

Nutzen Sie Exceptions um zu implementieren, dass die Funktion

- 1 zurückgibt, wenn die Datei nicht geöffnet werden konnte,
- 2 zurückgibt, wenn die Datei leer ist,
- 3, wenn die erste Zeile der Datei kein korrekt formatiertes Datum enthält und
- 4, falls irgendein anderer Fehler auftritt.

Hinweis: Auf Moodle finden Sie die Dateien `b09a2.py`, `valid.txt`, `invalid.txt` und `empty.txt` als Ausgangspunkt.

Hausaufgabe 3 (4 Punkte):

Entwerfen Sie geeignete Fälle für Black-Box-Testing der folgenden Funktion:

```
def add_val(wb, s, i):  
    '''  
    Erwartet ein Wörterbuch bestehend aus Schlüsseln vom Typ String und  
    Werten vom Typ Liste, einen String und einen Integer.  
    Falls der String als Schlüssel im Wörterbuch vorkommt wird überprüft,  
    ob der Integer in der zum Schlüssel gehörenden Liste enthalten ist.  
    Falls nein wird der Wert am Ende der Liste angefügt.  
    Falls ja passiert nichts.  
    Kommt der String nicht als Schlüssel im Wörterbuch vor, so wird eine  
    Fehlermeldung auf der Konsole ausgegeben.  
    In jedem Fall wird das, ggf. angepasste, Wörterbuch zurückgegeben.  
    '''
```

Geben Sie für jeden der Testfälle ein Beispiel für konkrete Eingaben an und begründen Sie die Wahl Ihrer Testfälle. Fälle in denen die Eingaben nicht wie beschrieben sind, müssen nicht behandelt werden. Auch Randfälle müssen nicht betrachtet werden.

*Die Testfälle sollten mit der Begründung als **.pdf-Datei** abgegeben werden.*

Hausaufgabe 4 (5 + 1 = 6 Punkte):

Entwerfen Sie geeignete Fälle für Glass-Box-Testing des Programms in der zur Verfügung gestellten Datei `b09a4.py`. Geben Sie für jeden der Testfälle ein Beispiel für konkrete Eingaben an und begründen Sie die Wahl Ihrer Testfälle.

Fügen Sie außerdem, an geeigneten Stellen im Programm, `print`-Befehle ein, die bei Ausführung Ihrer Testfälle die Abdeckung der Programmpfade zeigen.

Die Testfälle müssen, soweit möglich, Pfad-vollständig sein und Schleifen müssen gemäß der Richtlinien, welche in den Vorlesungsfolien beschrieben sind, behandelt werden.

*Die Testfälle sollten mit der Begründung als **.pdf-Datei** abgegeben werden. Das Programm mit eingefügten `print` Befehlen als **.py-Datei**.*

Präsenzaufgaben

Präsenzaufgabe 1:

Schreiben Sie eine Funktion `convertible_to_int`, welche eine Eingabe `x` erwartet und überprüft, ob `x` zu einem Integer (`int`) konvertiert werden kann. Falls ja, soll `True` ausgegeben werden, sonst `False`.

Präsenzaufgabe 2:

Schreiben Sie eine Funktion `key_value`, welche ein Wörterbuch, einen Integer `i` und eine Liste übergeben bekommt und folgendes umsetzt: Der i -te Eintrag der Liste wird als Schlüssel interpretiert und der zum Schlüssel zugehörige Wert des Wörterbuchs wird zurückgegeben.

Erweitern Sie dann Ihr Programm, sodass es folgende Werte vom Nutzer einliest: Die Länge einer Liste, einen Listenindex und die Liste selbst (als einzelne Einträge, entsprechend der eingegebenen Länge). Die Einträge der Liste sollen also Strings sein. Legen Sie zudem ein Wörterbücher an, mit welchem Sie Ihre Funktion testen können. Rufen Sie die Funktion `key_value` dann auf einem der selbst angelegten Wörterbücher, dem vom Nutzer erhaltenen Index und der vom Nutzer erhaltenen Liste auf.

Fangen Sie alle möglichen Exceptions, die auftreten könnten ab und geben Sie geeignete Fehlermeldungen auf der Konsole aus.

Präsenzaufgabe 3:

Betrachten Sie die folgende Spezifikation einer Funktion:

```
def move(L,s,i):  
    '''  
    Erwartet eine Liste von Strings, einen String und einen Integer.  
    Das erste Vorkommen des Strings in der Liste wird dann um i verschoben  
    d.h. i=2 bedeutet eine Verschiebung um 2 nach rechts.  
    Eine negative Zahl beschreibt eine Verschiebung nach links.  
    Die anderen Listenelemente werden entsprechend verschoben.  
    Wir interpretieren die Liste als verbunden, d.h. wenn wir den letzten  
    Eintrag um 1 nach rechts verschieben, wird er zum ersten Eintrag.  
    Die neue Liste wird dann zurückgegeben.  
    Falls der String nicht in der Liste ist, wird None zurückgegeben.  
    '''
```

Entwerfen Sie geeignete Fälle für Black-Box-Testing der Funktion. Betrachten Sie dabei auch Randfälle und diskutieren Sie welche Anforderungen zusätzlich gewünscht sein könnten. Fälle in denen die Eingaben nicht wie beschrieben sind, müssen nicht betrachtet werden.

Führen Sie das Black-Box-Testing an einer eigenen Implementierung der Funktionen durch.

Präsenzaufgabe 4:

Betrachten Sie den folgenden Programmcode:

```
x=int(input("Integer:"))
while x!=0:
    if x%2==0:
        x=0
    summe=0
    for i in range(x,100,2):
        summe+=i
    if summe>1131 and not summe==4950:
        x=x+2
    else:
        x=0
```

Entwerfen Sie Testfälle, sodass mit Glassbox-Testing, wenn möglich, alle Programmpfade abgedeckt werden. Dabei sollen Schleifen gemäß der Richtlinien, welche in den Vorlesungsfolien beschrieben sind, behandelt werden.

Verwenden Sie `print`-Anweisungen um sich zu versichern, dass Sie alle Programmpfade abdecken.